

# Recuperatorio del Parcial 2 — 2026

Sistemas Operativos / Sistemas Operativos y Redes

Tomás Rodeghiero

UNRC — Universidad Nacional de Río Cuarto

2026

## Resumen

Resolución completa del Recuperatorio del Parcial 2 del año 2026 de la cátedra de Sistemas Operativos (Lic. en Ciencias de la Computación, UNRC). El parcial consta de cinco ejercicios que cubren: (i) verdadero/falso sobre gestión de heap, segmentación, paginación, swapping y código PIE; (ii) construcción de una tabla de páginas a partir del layout lógico/físico de un proceso; (iii) memoria compartida entre dos procesos con la misma área mapeada a direcciones lógicas distintas; (iv) representación de un sistema de archivos basado en inodos con un archivo de varios bloques y su modificación tras un *append*; (v) descripción del flujo de *login* de un usuario hasta obtener un shell y cómo inhabilitarlo. Cada ejercicio se desarrolla con la estructura **Enunciado** → **Análisis** → **Desarrollo** → **Respuesta**.

## Índice

|                                                           |    |
|-----------------------------------------------------------|----|
| 1. Ejercicio 1 — V/F sobre memoria y kernel (2 pts)       | 1  |
| 2. Ejercicio 2 — Tabla de páginas (2 pts)                 | 4  |
| 3. Ejercicio 3 — Memoria compartida entre dos procesos    | 5  |
| 4. Ejercicio 4 — Filesystem basado en inodos              | 7  |
| 5. Ejercicio 5 — Login de un usuario y cómo inhabilitarlo | 8  |
| Resumen del recuperatorio                                 | 11 |

## 1. Ejercicio 1 — V/F sobre memoria y kernel (2 pts)

**Enunciado.** Determinar la verdad o falsedad de las siguientes sentencias. Justificar las falsas en forma breve, completa y con claridad.

### 1.A — Semi-compactación en heap de bloques variables

*“Un gestor de heap de bloques con tamaño variable puede usar la técnica de semi-compactación para evitar una fragmentación excesiva.”*

### Respuesta

**V.** Un gestor de heap con bloques de tamaño variable sufre *fragmentación externa*: la memoria libre queda partida en huecos pequeños no contiguos. Para mitigar esto, los gestores usan varias técnicas, entre ellas la **coalescencia (semi-compactación)** de bloques libres adyacentes:

- Cuando se libera un bloque, el gestor inspecciona si sus vecinos (en memoria) también están libres.
- Si alguno lo está, los *fusiona* en un único bloque más grande.
- Esto reduce la fragmentación sin necesidad de mover bloques ocupados (lo cual no es viable en `malloc` clásico porque rompería los punteros del usuario).

Es la técnica estándar de `dlmalloc`, `ptmalloc`, `jemalloc` y cualquier gestor moderno.

## 1.B — Bloques fijos y fragmentación

“En un heap con bloques de tamaño fijo ( $N$ ) no se produce ningún tipo de fragmentación.”

### Respuesta

**F.** Es cierto que con bloques de tamaño fijo **no hay fragmentación externa**: todos los bloques son intercambiables y, si hay  $k$  libres, alcanza con  $k$  pedidos para usarlos todos.

Pero **sí hay fragmentación interna**: si el usuario pide menos de  $N$  bytes, los bytes sobrantes del bloque (entre el tamaño pedido y  $N$ ) se desperdician. Por ejemplo, si  $N = 64$  y el usuario pide 8 bytes, 56 bytes quedan “adentro” del bloque sin uso.

Es exactamente el compromiso del modelo: ganamos en simplicidad de gestión (sin huecos externos) a costa de eficiencia (más memoria total se desperdicia internamente).

## 1.C — Cambio de contexto y registro de tabla de páginas

“En cada cambio de contexto el SO debe cambiar el registro de control que apunta a la tabla de páginas de la nueva tarea.”

### Respuesta

**V** (con un matiz importante). Cuando el SO conmuta de una tarea con un espacio de direcciones distinto, debe actualizar el registro de control que apunta a la *tabla de páginas raíz* del proceso entrante. En x86 es el registro `CR3`, en ARM `TTBR0/TTBR1`, en RISC-V `satp`. Cargar este registro hace que la MMU empiece a traducir direcciones según la nueva tabla y, en general, invalida (o etiqueta) la TLB.

**Matiz:** si la conmutación es entre dos *threads del mismo proceso* (que comparten espacio de direcciones), el registro de tabla de páginas **no cambia**; sólo se actualizan los registros de propósito general y el stack pointer. Por eso los context switches entre threads son más livianos.

El enunciado habla de “nueva tarea”, interpretado en el sentido clásico como un nuevo proceso (espacio de direcciones distinto), así que la afirmación es correcta.

## 1.D — Registros de descriptores de segmentos

“Comúnmente una CPU con segmentación ofrece un gran número de registros de descriptores de segmentos similar a una tabla de páginas.”

## Respuesta

**F.** Las CPUs con segmentación tienen un número **pequeño y fijo** de registros de segmento, no una cantidad grande similar a una tabla de páginas:

- En x86 hay exactamente **6 registros de segmento**: CS (código), DS (datos), ES, FS, GS (datos auxiliares), y SS (stack).

Los *descriptores de segmento* (la información completa: base, límite, permisos) sí viven en una tabla en memoria (GDT/LDT en x86), *análoga* a una tabla de páginas en ese aspecto. Pero los *registros* en CPU son pocos, y cada uno guarda solo un selector y un descriptor cacheado.

El error del enunciado es confundir el número de *registros* (pocos) con el número de *descriptores en la tabla*.

## 1.E — Swapping con paginado vs. con segmentación

“Con paginado es más fácil y eficiente implementar swapping que con segmentación.”

## Respuesta

**V.** Hay varias razones:

1. **Tamaño uniforme.** En paginado todas las páginas miden lo mismo (típicamente 4 KB). El swap-in/swap-out es I/O de tamaño fijo, predecible y eficiente. En segmentación los segmentos tienen tamaños variables, lo que complica la gestión.
2. **Sin fragmentación externa.** Cualquier frame libre sirve para alojar una página entrante; en segmentación hay que encontrar un hueco contiguo del tamaño exacto del segmento.
3. **Granularidad fina.** Con paginado se puede traer/sacar una sola página a la vez (*demand paging*); con segmentación habría que swapear un segmento entero, posiblemente de varios MB, aunque solo se necesiten unos KB.
4. **Hardware más simple.** El TLB y la MMU paginada son soluciones bien probadas; la segmentación pura es más rara en CPUs modernas (x86 mantiene segmentación por retrocompatibilidad pero usa paginado para todo).

## 1.F — Código del kernel y PIE

“El código del kernel no necesita compilarse como PIE (position independent code).”

## Respuesta

**V.** El código del kernel se carga siempre en **la misma dirección virtual fija**, conocida en tiempo de compilación. Por ejemplo, en Linux x86-64 el kernel vive en 0xFFFFFFFF80000000 y, en cada proceso, ese rango está mapeado a las páginas físicas del kernel. Como la dirección es estática, el código puede usar referencias absolutas sin necesidad de ser *position-independent*.

PIE existe precisamente porque los binarios de usuario *pueden* cargarse en distintas direcciones (típicamente por ASLR, para seguridad), y necesitan funcionar igual sin importar la base. Pero el kernel no necesita esa flexibilidad: vive en una única posición prefijada.

- **Matiz moderno:** para defenderse contra exploits, los kernels actuales soportan **KASLR** (*Kernel ASLR*), aleatorizando la base del kernel en cada arranque. En ese caso el código sí debe compilarse con técnicas similares a PIE (relocaciones tempranas). Pero *conceptualmente* no es PIE estándar, sino un mecanismo especial.

## 2. Ejercicio 2 — Tabla de páginas (2 pts)

**Enunciado.** Dado el siguiente esquema de la memoria de un proceso con páginas de 4 KB, dar la tabla de páginas.

| Dirección lógica       | Dirección física |
|------------------------|------------------|
| código [0x0000–0x1FFF] | 0x3000           |
| datos [0x2000–0x2FFF]  | 0x1000           |
| stack [0xF000–0xFFFF]  | 0xF000           |

### Análisis

#### Parámetros básicos.

- Tamaño de página: 4 KB =  $2^{12}$  bytes.
- Offset dentro de la página: 12 bits bajos.
- Número de página virtual (vpn) = bits altos de la dirección lógica.

#### Tamaño de cada área en páginas.

- Código [0x0000, 0x2000):  $0x2000 = 8 \text{ KB} = 2 \text{ páginas} \rightarrow$  vpns 0 y 1.
- Datos [0x2000, 0x3000):  $0x1000 = 4 \text{ KB} = 1 \text{ página} \rightarrow$  vpn 2.
- Stack [0xF000, 0x10000):  $0x1000 = 4 \text{ KB} = 1 \text{ página} \rightarrow$  vpn 15.

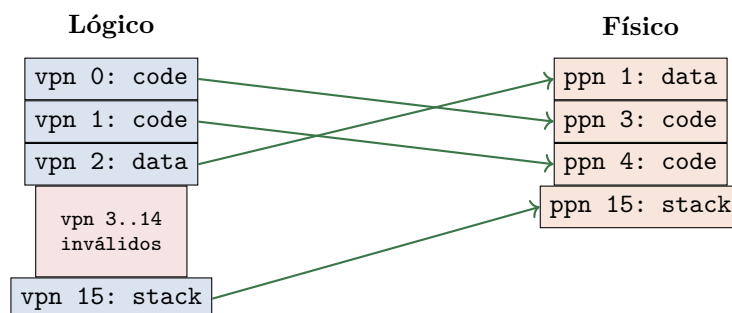
**Frames físicos.** Las direcciones físicas dadas indican la *dirección de inicio* en memoria de cada sección. Como el código ocupa 2 páginas a partir de 0x3000, las páginas físicas son 0x3000 (frame 3) y 0x4000 (frame 4).

código: ppn 3 (0x3000), ppn 4 (0x4000)  
datos: ppn 1 (0x1000)  
stack: ppn 15 (0xF000)

#### Tabla de páginas resultante

| vpn  | rango lógico     | ppn | V | R/W/X | Área              |
|------|------------------|-----|---|-------|-------------------|
| 0    | [0x0000, 0x0FFF] | 3   | 1 | R + X | código (página 1) |
| 1    | [0x1000, 0x1FFF] | 4   | 1 | R + X | código (página 2) |
| 2    | [0x2000, 0x2FFF] | 1   | 1 | R + W | datos             |
| 3–14 | [0x3000, 0xEFFF] | —   | 0 | —     | inválido (gap)    |
| 15   | [0xF000, 0xFFFF] | 15  | 1 | R + W | stack             |

#### Diagrama: traducción virtual $\rightarrow$ física



## Respuesta

Tabla de páginas resultante:

| vpn | ppn (V=1) |
|-----|-----------|
| 0   | 3         |
| 1   | 4         |
| 2   | 1         |
| 15  | 15        |

Las vpns 3 a 14 están **inválidas** ( $V = 0$ ): cualquier acceso a esas direcciones genera *page fault*.

### 3. Ejercicio 3 — Memoria compartida entre dos procesos

**Enunciado.** Dados dos procesos  $A$  y  $B$  con el layout del ejercicio anterior. Ambos hacen  $\text{ptr} = \text{shmem}(\text{key}, 5000)$  (crea un bloque de 5000 bytes compartido). En  $A$  retorna  $\text{ptr} = 0x5000$ ; en  $B$ ,  $\text{ptr} = 0x7000$ . Dar las tablas de páginas de ambos procesos.

#### Análisis

**Tamaño de la región compartida.** 5000 bytes con páginas de 4 KB:

$$\lceil 5000/4096 \rceil = \lceil 1.22 \rceil = 2 \text{ páginas.}$$

**Ubicación lógica en cada proceso.**

- **Proceso A:**  $\text{ptr} = 0x5000$ . La región ocupa  $[0x5000, 0x6388)$ , que abarca las páginas  $\text{vpn}$  5 y  $\text{vpn}$  6.
- **Proceso B:**  $\text{ptr} = 0x7000$ . La región ocupa  $[0x7000, 0x8388)$ , que abarca las páginas  $\text{vpn}$  7 y  $\text{vpn}$  8.

**Lo clave de la *shared memory*.** Aunque las  $\text{vpn}$  elegidas sean distintas en cada proceso, ambos *mapean a los mismos frames físicos*. Si el SO asigna los frames físicos 5 y 6 para la región compartida, entonces:

- En A:  $\text{vpn } 5 \rightarrow \text{ppn } 5$  y  $\text{vpn } 6 \rightarrow \text{ppn } 6$ .
- En B:  $\text{vpn } 7 \rightarrow \text{ppn } 5$  y  $\text{vpn } 8 \rightarrow \text{ppn } 6$ .

Una escritura del proceso A en  $0x5000$  es inmediatamente visible para el proceso B leyendo desde  $0x7000$ : ambas direcciones lógicas resuelven al mismo byte físico.

**Asignación de frames.** Asumimos que A y B son procesos distintos (no comparten ni código ni datos ni stack). Reutilizamos los frames de A según el Ej 2 y elegimos otros para B:

- A: código  $\rightarrow$  frames 3,4; datos  $\rightarrow$  frame 1; stack  $\rightarrow$  frame 15.
- Shared memory  $\rightarrow$  frames 5,6 (asignados por el SO en **shmem**).
- B: código  $\rightarrow$  frames 7,8; datos  $\rightarrow$  frame 9; stack  $\rightarrow$  frame 14.

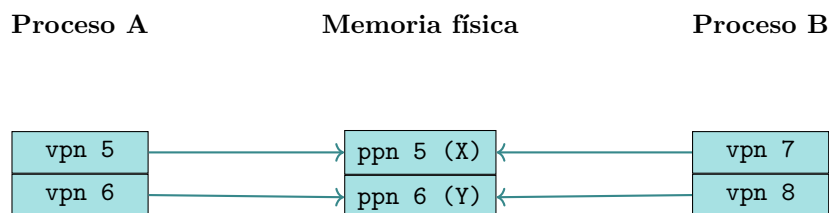
Tabla de páginas del proceso A

| vpn  | rango lógico     | ppn | V | Área                 |
|------|------------------|-----|---|----------------------|
| 0    | [0x0000, 0x0FFF] | 3   | 1 | código               |
| 1    | [0x1000, 0x1FFF] | 4   | 1 | código               |
| 2    | [0x2000, 0x2FFF] | 1   | 1 | datos                |
| 3, 4 | [0x3000, 0x4FFF] | —   | 0 | inválidas            |
| 5    | [0x5000, 0x5FFF] | 5   | 1 | compartida (frame X) |
| 6    | [0x6000, 0x6FFF] | 6   | 1 | compartida (frame Y) |
| 7–14 | [0x7000, 0xEFFF] | —   | 0 | inválidas            |
| 15   | [0xF000, 0xFFFF] | 15  | 1 | stack                |

Tabla de páginas del proceso B

| vpn  | rango lógico     | ppn | V | Área                 |
|------|------------------|-----|---|----------------------|
| 0    | [0x0000, 0x0FFF] | 7   | 1 | código               |
| 1    | [0x1000, 0x1FFF] | 8   | 1 | código               |
| 2    | [0x2000, 0x2FFF] | 9   | 1 | datos                |
| 3–6  | [0x3000, 0x6FFF] | —   | 0 | inválidas            |
| 7    | [0x7000, 0x7FFF] | 5   | 1 | compartida (frame X) |
| 8    | [0x8000, 0x8FFF] | 6   | 1 | compartida (frame Y) |
| 9–14 | [0x9000, 0xEFFF] | —   | 0 | inválidas            |
| 15   | [0xF000, 0xFFFF] | 14  | 1 | stack                |

Diagrama: las dos tablas apuntan a los mismos frames



### Respuesta

La región compartida ocupa 2 páginas físicas (frames  $X = 5$  e  $Y = 6$ ). En las dos tablas de páginas, las entradas correspondientes apuntan a esos mismos frames, pero a **vpns distintos**:

- Proceso A:  $\text{vpn } 5 \rightarrow \text{ppn } 5$ ,  $\text{vpn } 6 \rightarrow \text{ppn } 6$ .
- Proceso B:  $\text{vpn } 7 \rightarrow \text{ppn } 5$ ,  $\text{vpn } 8 \rightarrow \text{ppn } 6$ .

Una escritura de A en 0x5000 es inmediatamente visible para B en 0x7000: ambas direcciones lógicas resuelven al mismo byte físico vía la MMU.

### Idea intuitiva

La *shared memory* es el mecanismo IPC más eficiente: una vez mapeada, los accesos no pasan por el kernel; son lecturas/escrituras de memoria normales. La sincronización entre procesos (mutex, semáforos) se construye encima.

**Nota sobre fragmentación interna:** 5000 bytes ocupan 2 páginas = 8192 bytes. Se desperdician 3192 bytes en la segunda página. Es la fragmentación interna inherente a la paginación.

## 4. Ejercicio 4 — Filesystem basado en inodos

**Enunciado.** Dado un sistema de archivos basado en inodos con bloques de 512 bytes, donde el directorio raíz tiene un único archivo `README` que ocupa 900 bytes:

(A) Diagramar su imagen. (B) Mostrar la modificación luego de agregar (*append*) 500 bytes a `README`.

### Análisis: estructura de un FS de inodos

#### Componentes:

- **Superblock:** metadata global del FS (tamaño de bloque, número de inodos, etc.).
- **Tabla de inodos:** un inodo por archivo/directorio, con metadata (tipo, tamaño, permisos, punteros a bloques).
- **Bloques de datos:** contenido real de archivos y directorios.

#### Estructura típica de un inodo (estilo Unix clásico):

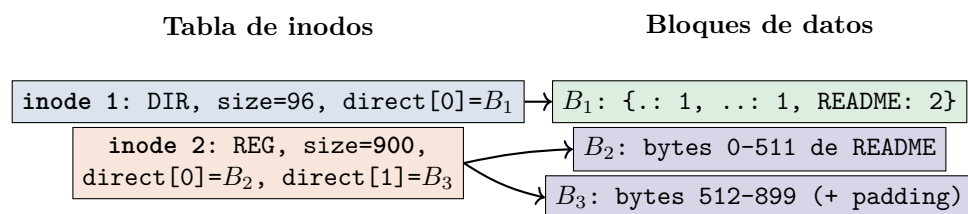
| Campo                              | Significado                                                      |
|------------------------------------|------------------------------------------------------------------|
| <code>i_mode</code>                | tipo (file, dir, link) + permisos                                |
| <code>i_size</code>                | tamaño del archivo en bytes                                      |
| <code>i_uid, gid</code>            | dueño y grupo                                                    |
| <code>i_atime, mtime, ctime</code> | timestamps                                                       |
| <code>direct[0..11]</code>         | punteros directos a 12 bloques de datos                          |
| <code>indirect</code>              | puntero a un bloque que contiene más punteros (indirecto simple) |
| <code>double, triple</code>        | indirección doble y triple para archivos grandes                 |

### 4.A — Imagen inicial

#### Cuentas previas.

- `README` ocupa 900 bytes.  $\lceil 900/512 \rceil = 2$  bloques. Bloque 1: bytes 0–511 (lleno). Bloque 2: bytes 512–899 (388 bytes ocupados, 124 bytes de relleno).
- Directorio raíz: contiene 3 entradas (`.`, `..`, `README`). Asumiendo 32 bytes por entrada:  $3 \times 32 = 96$  bytes, cabe en 1 bloque.

**Asignación.** Inodos 1 (root) y 2 (`README`). Bloques de datos:  $B_1$  (root dir),  $B_2$  y  $B_3$  (datos de `README`).



#### Descripción del estado inicial:

- inodo 1 (directorio raíz): `type = DIR, size = 96, direct[0] = B1`.
- Bloque  $B_1$  (contenido del directorio raíz): `{ . → inodo 1, .. → inodo 1, README → inodo 2 }`.
- inodo 2 (`README`): tipo `REG`, tamaño 900, punteros `direct[0] = B2` y `direct[1] = B3`; los demás punteros son nulos.
- Los bloques  $B_2$  y  $B_3$  contienen los 900 bytes del archivo (más relleno al final del segundo bloque).

## 4.B — Modificación tras append de 500 bytes

Nuevo tamaño:  $900 + 500 = 1400$  bytes.

$\lceil 1400/512 \rceil = 3$  bloques.

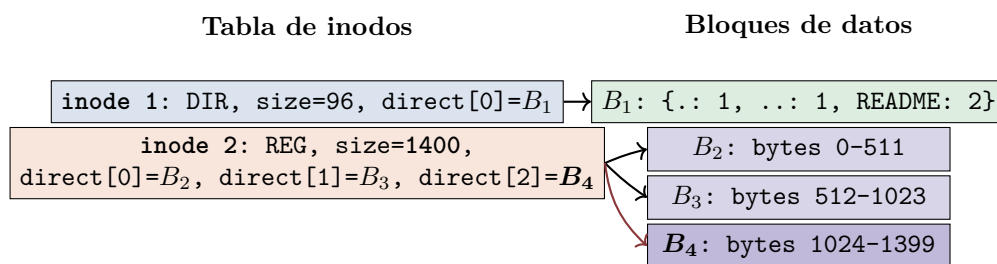
Ya tenemos 2 bloques ( $B_2$  y  $B_3$ ); hace falta uno más ( $B_4$ ) para alojar los bytes 1024–1399.

### Distribución del contenido tras append:

- $B_2$ : bytes 0–511 (lleno, no cambia).
- $B_3$ : bytes 512–1023 (antes tenía 512–899; ahora los bytes 900–1023 se completan con los primeros 124 bytes del append).
- $B_4$  (**nuevo**): bytes 1024–1399 (los restantes 376 bytes del append; bytes 1400–1535 quedan como padding).

### Cambios en la imagen del FS:

1. Se asigna un bloque libre  $B_4$  desde la *free list* del FS.
2. Se actualiza el inodo de README: `i_size` pasa de 900 a **1400**, y `direct[2]` pasa de *nulo* a  $B_4$ .
3. Los bloques  $B_2$  y  $B_3$  se reescriben con los datos actualizados.
4. Se actualiza `i_mtime` y `i_ctime`.
5. **El inodo del directorio raíz NO cambia**: ni el nombre ni el inodo de README cambiaron; sólo cambió su contenido.



### Respuesta

#### Cambios tras el *append* de 500 bytes a README:

- `i_size` del inodo 2 (README):  $900 \rightarrow 1400$ .
- Se agrega `direct[2] = B_4` (bloque nuevo).
- $B_3$  se completa (antes tenía 388 bytes; ahora 512).
- $B_4$  se asigna desde la *free list* y guarda los últimos 376 bytes (+ padding).
- **El inodo de la raíz no cambia**: el directorio sigue teniendo la misma entrada hacia el mismo inodo 2.

**Crucial:** el atributo del archivo (su inodo) es el que mueve el tamaño y agrega el bloque; el directorio raíz *no se entera del cambio*, sólo conoce el nombre y el número de inodo. Esto es lo que permite que *múltiples nombres* (hard links) compartan el mismo archivo.

## 5. Ejercicio 5 — Login de un usuario y cómo inhabilitarlo

**Enunciado.** Describir brevemente el proceso de login de un usuario en un sistema hasta que obtiene un shell. ¿Cómo podríamos inhabilitar su login?

### Análisis: anatomía del login

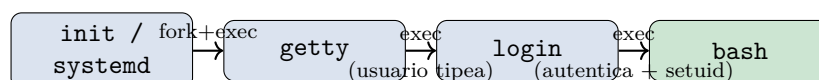
El login es la cadena de pasos que transforma un usuario humano sentado frente a una terminal en un *shell* ejecutando con sus privilegios. Recorre desde `init/systemd` hasta la línea de comandos del usuario.



## Proceso paso a paso

1. **Arranque del sistema y init.** Al iniciar la máquina, el kernel monta el FS raíz y ejecuta `init` (PID 1: `systemd` en Linux moderno). `init` es el padre de todos los procesos de usuario.
2. **Lanzamiento de `getty` en cada TTY.** Para cada terminal (TTY, p.ej. `tty1`, `tty2...`), `systemd` lanza un proceso `getty` (*get tty*) que abre el dispositivo, configura velocidad/paridad e imprime el prompt “login:”.
3. **Lectura del nombre de usuario.** `getty` lee la línea ingresada. Cuando el usuario teclea su nombre y presiona Enter, `getty` hace `exec("login", username, ...)`: reemplaza su imagen por el binario `login`, conservando el mismo PID y la TTY ya abierta.
4. **Lectura de la contraseña.** `login` imprime “Password:” y lee con echo *deshabilitado* (vía `ioctl` sobre el `termios`).
5. **Autenticación vía PAM** (*Pluggable Authentication Modules*). `login` delega la validación a PAM, que:
  - a) Busca al usuario en `/etc/passwd` (campo: login, UID, GID, GECOS, home, shell).
  - b) Lee el hash de `/etc/shadow`.
  - c) Aplica la misma función hash (con salt) a la contraseña ingresada y compara.
  - d) Verifica reglas adicionales: cuenta no caducada, horario permitido, no bloqueada por `pam_faillock` tras varios intentos fallidos, etc.
6. **Si autentica correctamente:** `login` configura el proceso para que “sea” el usuario:
  - `setgid(gid)` y `setgroups(...)` para grupo primario y suplementarios.
  - `setuid(uid)`, perdiendo definitivamente los privilegios root.
  - `chdir(home)` al directorio personal.
  - `setenv`: HOME, USER, LOGNAME, SHELL, PATH, TERM, MAIL.
7. **Exec del shell.** Como último paso, `login` hace `exec(pw_shell)` con el shell del usuario (campo 7 de `/etc/passwd`, e.g. `/bin/bash`). El proceso ahora corre `bash` como ese usuario.
8. **El shell arranca:** lee `/etc/profile`, `~/.bash_profile` o `~/.bashrc`, exporta variables, configura alias y muestra el prompt. El usuario está adentro.

## Diagrama del flujo



**Observación clave:** el shell del usuario hereda el PID del `getty` original (a través de los sucesivos `exec`). El usuario nunca causa un `fork`; toda la cadena son reemplazos de imagen sobre el mismo proceso.

## Cómo inhabilitar el login de un usuario

Hay varias técnicas, en orden creciente de severidad:

**Opción A: cambiar el shell del usuario a `nologin`.** La técnica estándar. En `/etc/passwd`, el séptimo campo es el shell. Si se pone `/sbin/nologin` o `/usr/sbin/nologin`:

```
sudo usermod -s /sbin/nologin USUARIO
```

El binario `/sbin/nologin` imprime el texto de `/etc/nologin.txt` (“*This account is currently not available*”) y termina con `exit(1)`. Resultado: el usuario *puede autenticarse* pero al hacer `exec()` del shell el proceso muere instantáneamente y se vuelve al prompt.

**Opción B: bloquear la contraseña.** Modificar el hash en `/etc/shadow` para que ninguna entrada del usuario pueda hashearse al mismo valor:

```
sudo passwd -l USUARIO # antepone "!" al hash
# 0
sudo usermod -L USUARIO
```

PAM ya no autentica y el login falla en el paso 5. Reversible con `passwd -u USUARIO`.

**Opción C: expirar la cuenta.**

```
sudo chage -E 0 USUARIO # cuenta expirada hace tiempo
sudo usermod --expiredate 1 USUARIO
```

PAM detecta la expiración y rechaza el login.

**Opción D: borrar al usuario.**

```
sudo userdel USUARIO # mantiene home
sudo userdel -r USUARIO # borra home también
```

Definitivo. No tiene reverso.

**Opción E: regla PAM.** En `/etc/pam.d/login` agregar:

```
auth required pam_listfile.so onerr=succeed item=user \
    sense=deny file=/etc/login.deny
```

y poner el nombre del usuario en `/etc/login.deny`.

### Respuesta

El login de un usuario es la cadena **init/systemd** → **getty** → **login** → **shell**, encadenada por sucesivos `exec()` sobre el mismo PID, con autenticación vía PAM contra `/etc/passwd` y `/etc/shadow`, y un `setuid/setgid` al final que baja los privilegios al usuario.

**Para inhabilitar** el login de un usuario, las técnicas estándar son:

1. `sudo usermod -s /sbin/nologin USUARIO` (cambia el shell a uno que sale inmediatamente). *La más usada.*
2. `sudo passwd -l USUARIO` (bloquea la contraseña).
3. `sudo chage -E 0 USUARIO` (expira la cuenta).
4. `sudo userdel USUARIO` (borrado total).
5. Reglas en `pam.d` (`pam_listfile` con un archivo de denegados).

La elección depende de si se quiere **reversibilidad** (opciones 1, 2, 3, 5) o no (opción 4), y de si el usuario tiene servicios automatizados (cron, sshd con keys) que también deban desactivarse.

## Resumen del recuperatorio

| # | Tema               | Respuesta sintética                                                                                                                 |
|---|--------------------|-------------------------------------------------------------------------------------------------------------------------------------|
| 1 | V/F memoria/kernel | A:V (coalescing), B:F (frag. interna), C:V (con matiz threads), D:F (pocos registros), E:V (uniformidad), F:V (kernel en dir. fija) |
| 2 | Tabla de páginas   | vpn 0→3, 1→4, 2→1, 15→15; resto V=0                                                                                                 |
| 3 | Shared memory      | A: vpn 5,6 → ppn 5,6; B: vpn 7,8 → ppn 5,6 (mismos frames físicos)                                                                  |
| 4 | FS de inodos       | (A) inode README size=900, direct[0,1]; (B) tras append size=1400, direct[2]=B <sub>4</sub> nuevo                                   |
| 5 | Login y bloqueo    | init→getty→login→shell con PAM y setuid. Bloquear con <code>usermod -s /sbin/nologin, passwd -l, chage -E 0</code> , etc.           |